How to Dna2prot

`Dna2prot.pm` is a Perl module to process DNA and protein sequences for molecular biology. Below is a sample test script, `test_dna2prot.pl`, to process `PA_DNA.txt`, a text file with a DNA sequence. The script opens the DNA containing file, converts it to fasta format and resaves it. Then it searches the DNA sequence for the protein sequence encoding the largest open reading frame (orf), `find_largest_orf`. Finally, the script saves the record file, `PA_DNA_record.txt`, containing protein parameter information and the DNA and protein sequences encoding the orf:

```
use Dna2prot;
my $dna2prot =  Dna2prot
             -> new()
             -> open_dna('PA_DNA.txt')
             -> save_dna('PA_DNA.txt')
             -> find_largest_orf;
   $dna2prot -> protein_name('PA');
   $dna2prot -> save_record('PA_DNA_record.txt');
```

The record file

This file format of the record file, `PA_DNA_record.txt`, is as follows. Protein parameters are given for the protein sequence, first fasta protein sequence, encoded in the DNA, given as the second fasta DNA sequence. The long sequence strings of the DNA and protein are cut off for brevity. The greater than signs, '>', denote the names of the protein and DNA sequences. The line `$dna2prot -> protein_name('PA')` sets the name of the protein to 'PA' in the module. Note in this file format comments and annotations may be added as long as the line is commented out with a '#' sign.

```
# Protein name: PA
# Protein length: 764 residues
# Molecular weight: 85838.2 Da
# Isoelectric point (pI): 5.89
# Extinction coefficient: 80220 M-1 cm-1

>PA
MKKRKVLIPLMALSTILVSSTGNLEVIQAEVKQENRLLNESESSSQGLLGYYFSDLNFQAPMVVTSSTTGDLSIPS…

>PA_DNA
ATGAAAAAACGAAAAGTGTTAATACCATTAATGGCATTGTCTACGATATTAGTTTCAAGCACAGGTAATTTAGAGG…
```

A simple DNA file

The other file used, `PA_DNA.txt`, is a simple fasta format file, which can be opened by the `open_dna` method. (A similar method exists for simple fasta protein sequence files, called `open_protein`:

```
>PA_DNA
TAAATTCTTTTTTATGTTATATATTTATAAAAGTTCTGTTTAAAAAGCCAAAAATAAATAATTATCTCTTTTTATTTAT
ATTATATTGAAACTAAAGTTTATTAATTTCAATATAATATAAATTTAATTTTATACAAAAAGGAGAACGTATATGAAAA
AACGAAAAGTGTTAATACCATTAATGGCATTGTCTACGATATTAGTTTCAAGCACAGGTAATTTAGAGGTGATTCAGGC
AGAAGTTAAACAGGAGAACCGGTTATTAAATGAATCAGAATCAAGTTCCCAGGGGTTACTAGGATACTATTTTAGTGAT
…
```

Saving sequence data

Simple DNA and protein sequence data can be saved with `save_dna` and `save_protein`. These methods can take two arguments, the filename and the sequence to be saved. If the respective protein or DNA sequence is not passed as an argument, then the current DNA or protein

sequence is used. These sequences are accessed by the properties, `current_protein_sequence` and `current_dna_sequence`. Accessing these properties and methods are exemplified in the following script:

```
use Dna2prot;
my $protein  =  'ADFGHIKVRWFH'; #use one-letter amino acid abbreviations
my $dna2prot =  Dna2prot -> new();
   $dna2prot -> current_protein_sequence($protein);
   $dna2prot -> protein_name('my_peptide');
   $dna2prot -> save_protein('my_protein.txt');
```

Calculating protein parameters

The next test script, `test_dna2prot_2.pl`, shows how calculated protein parameters can be accessed from the module.

```
use Dna2prot;
my $dna2prot =  Dna2prot
             -> new()
             -> open_protein('PA_protein.txt')
             -> save_protein('PA_protein.txt')
             -> calculate_protein_parameters;
my $isoelectric_point = $dna2prot -> isoelectric_point;
print "PA's isoelectric point is $isoelectric_point\n";
   $dna2prot -> open_dna('PA_DNA.txt')
             -> save_record('PA_DNA_record.txt');
```

The method, `calculate_protein_parameters`, calculates protein length, molecular weight, extinction coefficient, and isoelectric point on the current protein sequence. The value for the isoelectric point can be access from the property, `isoelectric_point`. There are similar properties for the other parameters, `extinction_coefficient`, `molecular_weight`, and `protein_length`. All these parameters are saved as a commented file header in the record file, `PA_DNA_record.txt`.

Methods to calculate individual protein parameters are also in the module. These include, `calculate_protein_length`, `calculate_isoelectric_point`, `calculate_protein_length`, and `calculate_extinction_coefficient`, which are self-explanatory. These can take the argument of a valid one-letter fasta format protein sequence or no argument. When no argument is given for all these methods, the `current_protein_sequence` is used.

Aligning and joining two DNA sequencing reads

DNA sequencing reads can be aligned and joined to help find open reading frames for the protein sequence using the method, `align_and_join`. This method takes two arguments, the two DNA sequences (in fasta format) to be aligned and joined. The next somewhat laborious test script, `test_dna2prot_3.pl`, shows how this may be done:

```
use Dna2prot;
my $dna2prot1 =  Dna2prot
              -> new()
              -> open_dna('PA_DNA_read1.txt')
              -> save_dna('PA_DNA_read1.txt');

my $dna1 = $dna2prot1 -> current_dna_sequence;

my $dna2prot2 =  Dna2prot
```

```
              -> new()
              -> open_dna('PA_DNA_read2.txt')
              -> save_dna('PA_DNA_read2.txt');

my $dna2 = $dna2prot2 -> current_dna_sequence;

my $dna2prot3 =  Dna2prot
              -> new()
              -> align_and_join($dna1, $dna2);

my $aligned_joined_dna = $dna2prot3 -> current_dna_sequence;

$dna2prot3    -> find_largest_orf
              -> save_dna('PA_DNA_joined.txt')
$dna2prot3    -> protein_name('PA');
$dna2prot3    -> dna_name('PA_DNA');
$dna2prot3    -> save_record('PA_DNA_record_2.txt');
```

When `align_and_join` is coupled with `find_largest_orf`, the encoded protein sequence can be identified and translated readily.

A less laborious script, `test_dna2prot_4.pl`, for aligning and joining two sequence read files is as follows:

```
use Dna2prot;
my $dna2prot = Dna2prot
          -> new
          -> align_and_join_files('PA_DNA_read1.txt', 'PA_DNA_read2.txt');

$dna2prot -> find_largest_orf
          -> save_dna('PA_DNA_joined.txt');
$dna2prot -> protein_name('PA');
$dna2prot -> dna_name('PA_DNA');
$dna2prot -> save_record('PA_DNA_record_3.txt');
```

The `align_and_join_files` method takes two filenames of the two sequencing reads as arguments and is pretty self-explanatory, where the aligned and joined sequence is stored in the property, `current_dna_sequence`.

The verbose translate output style

Sometimes to manually parse through a protein sequence (and corresponding DNA sequence) requires a more explicit file format. The following script, `test_dna2prot_5.pl`, shows the `verbose_translate` method.

```
use Dna2prot;
my $dna2prot = Dna2prot
  -> new()
  -> open_dna('PA_DNA.txt')
  -> find_largest_orf
  -> verbose_translate
  -> save_textbox('PA_verbose_translation.txt');
```

The method `verbose_translate` can take the parameters for the fasta DNA sequence, the amino acid numbering offset, the position translation starts, and the characters per line in the output. A truncated output of `PA_verbose_translation.txt` is as follows:

```
|1  |2  |3  |4  |5  |6  |7  |8  |9  |10 |11 |12 |13 |14 |15 |16 |17 |18 |19 |20 |
|MET|LYS|LYS|ARG|LYS|VAL|LEU|ILE|PRO|LEU|MET|ALA|LEU|SER|THR|ILE|LEU|VAL|SER|SER|
|ATG|AAA|AAA|CGA|AAA|GTG|TTA|ATA|CCA|TTA|ATG|GCA|TTG|TCT|ACG|ATA|TTA|GTT|TCA|AGC|

|21 |22 |23 |24 |25 |26 |27 |28 |29 |30 |31 |32 |33 |34 |35 |36 |37 |38 |39 |40 |
|THR|GLY|ASN|LEU|GLU|VAL|ILE|GLN|ALA|GLU|VAL|LYS|GLN|GLU|ASN|ARG|LEU|LEU|ASN|GLU|
|ACA|GGT|AAT|TTA|GAG|GTG|ATT|CAG|GCA|GAA|GTT|AAA|CAG|GAG|AAC|CGG|TTA|TTA|AAT|GAA|

|41 |42 |43 |44 |45 |46 |47 |48 |49 |50 |51 |52 |53 |54 |55 |56 |57 |58 |59 |60 |
|SER|GLU|SER|SER|SER|GLN|GLY|LEU|LEU|GLY|TYR|TYR|PHE|SER|ASP|LEU|ASN|PHE|GLN|ALA|
|TCA|GAA|TCA|AGT|TCC|CAG|GGG|TTA|CTA|GGA|TAC|TAT|TTT|AGT|GAT|TTG|AAT|TTT|CAA|GCA|
…
```

The output text of `verbose_translate` is first collected in a property called `textbox`. The textbox can then be saved with the method `save_textbox`, which takes a filename as its only argument.

Restriction enzyme mapping a DNA sequence

A staple of molecular biology is the restriction enzyme map of a DNA sequence. The method `find_restriction_sites` finds and reports the complete restriction enzyme map of a DNA sequence to the `textbox`. For example, the script, `test_dna2prot_6.pl`, shows:

```
use Dna2prot;

my $dna2prot = Dna2prot
  -> new()
  -> open_dna('PA_DNA.txt')
  -> find_restriction_sites
  -> save_textbox('PA_restriction_sites.txt');
```

Again, the `textbox` data are saved with the method, `save_textbox`. Here is a truncated file output of the restriction enzyme map for the above example:

```
# Restriction Enzyme Map
AccI GT/MKAC 190
AclI AA/CGTT 2019
AflIII A/CRYGT 1024 1365
AgeI A/CCGGT 255
AluI AG/CT 535 1009 1423
ApaBI GCANNNNN/TGC 1496
ApoI R/AATTY 2 120 319 588 1219 1686 1883 2406
AseI AT/TAAT 101 178 527 2096 2360
AvaII G/GWCC 645 752
BanI G/GYRCC 1486
BbeI GGCGC/C 1486
BclI T/GATCA 1240
BfaI C/TAG 296 375 678 1137 1256 1767 2235 2367 2385 2454
BisI GC/NGC 1007 1754
…
```

Note the restriction enzyme mapping routine uses a database file, called `rebase.txt`. You must keep it in the same directory with the `Dna2prot.pm` module file. This file concisely contains all the restriction enzyme data.

Calculating primer properties for a DNA sequence

When designing primer sequences, the melting point and potentially other primer parameters need to be calculated. The method, `calculate_primer_tm`, calculates these properties and outputs them to a textbox, which can be saved. An example script file, `test_dna2prot_7.pl`, shows how to use the method:

```
use Dna2prot;
# Primer has three mismatches (8.6% mismatch) from PA_DNA
my $dna2prot = Dna2prot
  -> new()
  -> open_dna('PA_DNA.txt')
  -> calculate_primer_tm('GTCAAAAATAAAACTACTATTCTTTCACCATGGATT')
  -> save_textbox('PA_primer.txt');
```

First, the script opens a DNA sequence. Then a primer sequence is passed as the first argument to `calculate_primer_tm` method. The method also takes a second argument which is the parent DNA sequence. This sequence is the current DNA sequence when it is omitted as in the above case. The parent sequence is used to determine with percent mismatch, which is part of the Tm calculation. Below is an example of the `textbox` output for the `calculate_primer_tm` method used in the above script:

```
PA_DNA.F|5'-GTCAAAAATAAAACTACTATTCTTTCACCATGGATT-3'
PA_DNA.R|5'-AATCCATGGTGAAAGAATAGTAGTTTTATTTTTGAC-3'
Primer length: 36
Melting midpoint (Tm): 65.8°C
Percent GC: 27.8
Percent mismatch: 8.3
```

For reference, the equation used in the Tm calculation is:
Tm = 81.5 + 0.41(%GC) - 675/N - % mismatch, where N is the total number of bases.

Site-directed mutagenesis and Quikchange primer pair design

Site-directed mutagenesis can be performed on the DNA and protein sequences with the `site_directed_mutagenesis` method. The method makes a point mutation and then creates a Quikchange mutagenesis primer pair with optimal GC clamping and Tm. The method takes four arguments.

```
$dna2prot -> site_directed_mutagenesis($amino_acid_position_to_mutate,
$amino_acid_mutation, $codon_mutation, $target_primer_tm);
$dna2prot -> save_textbox('PA_primer_design.txt');
```

But really only two arguments are needed, the residue position to be mutated and the desired replacement amino acid in either one-letter or three-letter abbreviation. The following script, `test_dna2prot_8.pl`, shows an example of mutating Gly 70 to Tyr:

```
use Dna2prot;
my $dna2prot = Dna2prot
  -> new()
  -> open_dna('PA_DNA.txt')
  -> find_largest_orf();
 $dna2prot -> protein_name('PA');
 $dna2prot -> site_directed_mutagenesis(70, 'TYR')
```

```
  -> save_textbox('PA_mutant_primer.txt');
```

The Quikchange primer pair design is outputted to a `textbox`, which can be saved as described above. Here is the output in the `PA_mutant_primer.txt` file:

```
PA_G70Y.F|5'-CATGGTGGTTACTTCTTCTACTACATACGATTTATCTATTCCTAGTTCTGAGTTAG-3'
PA_G70Y.R|5'-CTAACTCAGAACTAGGAATAGATAAATCGTATGTAGTAGAAGAAGTAACCACCATG-3'
Primer length: 56 bases
Melting midpoint (Tm): 78.7°C
Percent GC: 35.7
Percent mismatch: 5.4
```

The default target melting midpoint (Tm) is set to 78 °C, the manufacturer's recommendation. You can change the Tm by passing a different temperature argument to the `site_directed_mutagenesis` method.

Other miscellaneous DNA and protein sequence methods

The method `reverse_complement` takes one argument, a DNA sequence, and converts that sequence to the reverse complement and stores the sequence in `current_dna_sequence`.

```
Use Dna2prot;
my $dna = 'ATGGGCTAAGACAGTCGCTGACTGCATGCTGCA';
my $dna2prot = Dna2prot -> new();
$dna2prot -> reverse_complement($dna);
my $rc_dna = $dna2prot -> current_dna_sequence;
```

The method `fasta_dna` removes all non-DNA characters, numbers, and whitespace from a DNA sequence. It takes one argument, a DNA sequence. The method stores the fasta sequence in `current_dna_sequence`.

The method `fasta_protein` removes all non-amino acid characters and whitespace from a protein sequence. It takes one argument, a protein sequence. The method stores the fasta sequence in `current_protein_sequence`.

The method `translate` converts a DNA sequence to a protein sequence. Unlike the method `find_largest_orf`, the method `translate` does not search for a start codon or stop codon. It just translates the codons to their corresponding amino acids and stop codons. The method takes two arguments. The first argument is a DNA sequence. The second argument is DNA position to start the translation from. If the DNA sequence is omitted, then the `current_dna_sequence` is used. If position argument is omitted, the translation starts at DNA base zero, which is the first base.

```
use Dna2prot;
my $dna = 'AAAATGGGCTAAGACATAGGTCGCTGACTGCATGCTGCA';
my $dna2prot = Dna2prot -> new();
$dna2prot -> translate($dna, 1);
my $peptide = $dna2prot -> current_protein_sequence;
print "Amino acid sequence is $peptide\n";
```

The output of this script is:

```
Amino acid sequence is KWAKTZVADCML
```

The method `translate` also creates arrays of the amino acids, codons, and amino acid numbers. These can be accessed as properties of the module using `codons`, `amino_acids`, and `amino_acid_numbers`. For example,

```
use Dna2prot;
my $dna = 'AAAATGGGCTAAGACATAGGTCGCTGACTGCATGCTGCA';
my $dna2prot = Dna2prot -> new();
$dna2prot -> translate($dna, 1);
my @codons = $dna2prot -> codons;
my @amino_acid_numbers = $dna2prot -> amino_acid_numbers;
print "The number $amino_acid_numbers[2] codon is $codons[2]\n";
```

The output is

```
The number 3 codon is GCT
```

Note the third amino acid or codon is index 2 in the arrays, since those start at index zero and amino acids start at index 1 unless there is an amino acid offset (see below).

Offset corrects amino acid numbering

Sometimes a protein construct has an amino-terminal His6 tag or secretion sequence or other non-wild-type residues. You can offset the amino acid numbers with the `amino_acid_numbers_offset` property. When `translate` method is called, then the `amino_acid_numbers` will be corrected by adding the offset set in the `amino_acid_numbers_offset` property. Typically a negative value offset is needed to make the wild-type portion of the protein the correct numbering. See the test script, `test_dna2prot_9.pl`.

```
use Dna2prot;
my $dna2prot = Dna2prot
           -> new()
           -> open_dna('PA_DNA.txt')
           -> find_largest_orf
           -> verbose_translate
           -> save_textbox('PA_verbose_translation.txt');
 $dna2prot -> amino_acid_numbers_offset(-20);
 $dna2prot -> verbose_translate
           -> save_textbox('PA_verbose_translation_offset.txt');
```

The output in the second verbose translate after `amino_acid_numbers_offset` was set is saved in `PA_verbose_translation_offset.txt`.

```
|-19|-18|-17|-16|-15|-14|-13|-12|-11|-10|-9 |-8 |-7 |-6 |-5 |-4 |-3 |-2 |-1 |0  |
|MET|LYS|LYS|ARG|LYS|VAL|LEU|ILE|PRO|LEU|MET|ALA|LEU|SER|THR|ILE|LEU|VAL|SER|SER|
|ATG|AAA|AAA|CGA|AAA|GTG|TTA|ATA|CCA|TTA|ATG|GCA|TTG|TCT|ACG|ATA|TTA|GTT|TCA|AGC|

|1  |2  |3  |4  |5  |6  |7  |8  |9  |10 |11 |12 |13 |14 |15 |16 |17 |18 |19 |20 |
|THR|GLY|ASN|LEU|GLU|VAL|ILE|GLN|ALA|GLU|VAL|LYS|GLN|GLU|ASN|ARG|LEU|LEU|ASN|GLU|
|ACA|GGT|AAT|TTA|GAG|GTG|ATT|CAG|GCA|GAA|GTT|AAA|CAG|GAG|AAC|CGG|TTA|TTA|AAT|GAA|

|21 |22 |23 |24 |25 |26 |27 |28 |29 |30 |31 |32 |33 |34 |35 |36 |37 |38 |39 |40 |
|SER|GLU|SER|SER|SER|GLN|GLY|LEU|LEU|GLY|TYR|TYR|PHE|SER|ASP|LEU|ASN|PHE|GLN|ALA|
|TCA|GAA|TCA|AGT|TCC|CAG|GGG|TTA|CTA|GGA|TAC|TAT|TTT|AGT|GAT|TTG|AAT|TTT|CAA|GCA|

|41 |42 |43 |44 |45 |46 |47 |48 |49 |50 |51 |52 |53 |54 |55 |56 |57 |58 |59 |60 |
|PRO|MET|VAL|VAL|THR|SER|SER|THR|THR|GLY|ASP|LEU|SER|ILE|PRO|SER|SER|GLU|LEU|GLU|
|CCC|ATG|GTG|GTT|ACT|TCT|TCT|ACT|ACA|GGG|GAT|TTA|TCT|ATT|CCT|AGT|TCT|GAG|TTA|GAA|
```

```
…
```

As you can see, the residue numbering was offset by -20, and the old Gly70 is now Gly50.

Offset renumbering of amino acids is useful when performing site-directed mutagenesis. For example, see the following script, `test_dna2prot_10.pl`.

```
use Dna2prot;
my $dna2prot =  Dna2prot
             -> new()
             -> open_dna('PA_DNA.txt')
             -> find_largest_orf;
$dna2prot    -> amino_acid_numbers_offset(-20);
$dna2prot    -> protein_name('PA');
$dna2prot    -> site_directed_mutagenesis(50, 'PHE')
             -> save_textbox('PA_mutagenesis_primer_G50F.txt')
             -> transfer_mutant_to_current
             -> verbose_translate
             -> save_textbox('PA_verbose_translation_G50F.txt');
```

The first output saved to the textbox is in `PA_mutagenesis_primer_G50F.txt`.

```
PA_G50F.F|5'-CCATGGTGGTTACTTCTTCTACTACATTTGATTTATCTATTCCTAGTTCTGAGTTAG-3'
PA_G50F.R|5'-CTAACTCAGAACTAGGAATAGATAAATCAAATGTAGTAGAAGAAGTAACCACCATGG-3'
Primer length: 57 bases
Melting midpoint (Tm): 78.7°C
Percent GC: 35.1
Percent mismatch: 5.3
```

You can check the verbose translation of the mutant sequence if you transfer the mutant to the current sequence using the method `transfer_mutant_to_current`.

Here is the second output of the saved textbox file, `PA_verbose_translation_G50F.txt`:

```
|-19|-18|-17|-16|-15|-14|-13|-12|-11|-10|-9 |-8 |-7 |-6 |-5 |-4 |-3 |-2 |-1 |0  |
|MET|LYS|LYS|ARG|LYS|VAL|LEU|ILE|PRO|LEU|MET|ALA|LEU|SER|THR|ILE|LEU|VAL|SER|SER|
|ATG|AAA|AAA|CGA|AAA|GTG|TTA|ATA|CCA|TTA|ATG|GCA|TTG|TCT|ACG|ATA|TTA|GTT|TCA|AGC|

|1  |2  |3  |4  |5  |6  |7  |8  |9  |10 |11 |12 |13 |14 |15 |16 |17 |18 |19 |20 |
|THR|GLY|ASN|LEU|GLU|VAL|ILE|GLN|ALA|GLU|VAL|LYS|GLN|GLU|ASN|ARG|LEU|LEU|ASN|GLU|
|ACA|GGT|AAT|TTA|GAG|GTG|ATT|CAG|GCA|GAA|GTT|AAA|CAG|GAG|AAC|CGG|TTA|TTA|AAT|GAA|

|21 |22 |23 |24 |25 |26 |27 |28 |29 |30 |31 |32 |33 |34 |35 |36 |37 |38 |39 |40 |
|SER|GLU|SER|SER|SER|GLN|GLY|LEU|LEU|GLY|TYR|TYR|PHE|SER|ASP|LEU|ASN|PHE|GLN|ALA|
|TCA|GAA|TCA|AGT|TCC|CAG|GGG|TTA|CTA|GGA|TAC|TAT|TTT|AGT|GAT|TTG|AAT|TTT|CAA|GCA|

|41 |42 |43 |44 |45 |46 |47 |48 |49 |50 |51 |52 |53 |54 |55 |56 |57 |58 |59 |60 |
|PRO|MET|VAL|VAL|THR|SER|SER|THR|THR|PHE|ASP|LEU|SER|ILE|PRO|SER|SER|GLU|LEU|GLU|
|CCC|ATG|GTG|GTT|ACT|TCT|TCT|ACT|ACA|TTT|GAT|TTA|TCT|ATT|CCT|AGT|TCT|GAG|TTA|GAA|
```

The `transfer_mutant_to_current` method is also useful if you want to make site-directed mutants in a previously mutated sequence for a multi-site mutations construct.

Reverse translation

A protein sequence can be used to generate a possible reverse translated DNA sequence using the method `reverse_translate`. The method only takes a protein sequence as an argument. If protein sequence is omitted, then the `current_protein_sequence` is used. The reverse translated

DNA sequence is stored in `current_dna_sequence`, and the property arrays `amino_acids`, `codons`, and `amino_acid_numbers` are created. Note `amino_acid_numbers` is offset if the `amino_acid_numbers_offset` property is non-zero.

Here is an example script, `test_dna2prot_11.pl`.

```
use Dna2prot;
my $protein  =  'MGIKLPWRTEFGSTZ';
my $dna2prot =  Dna2prot
             -> new()
             -> reverse_translate($protein)
             -> verbose_translate
             -> save_textbox('PA_verbose_translation_reverse_translate.txt');
```

The following is the verbose translation of the generated DNA sequence in the output file, `PA_verbose_translation_reverse_translate.txt`.

```
|1  |2  |3  |4  |5  |6  |7  |8  |9  |10 |11 |12 |13 |14 |15 |
|MET|GLY|ILE|LYS|LEU|PRO|TRP|ARG|THR|GLU|PHE|GLY|SER|THR|STP|
|ATG|GGA|ATT|AAA|CTA|CCA|TGG|CGA|ACC|GAA|TTC|GGG|TCA|ACA|TAA|
```

Finding and Replacing Self-matching DNA Sequences

Sometimes a DNA sequence has self-similar regions. Such similar regions can create problems when primers anneal improperly, especially during gene synthesis methods. The self-matching DNA sequences can be removed with the `find_and_replace_self_matches` method. The method finds regions greater than a user-selected maximum length that are matching; it then replaces the regions with silent mutations. The method takes three arguments: the DNA sequence, the reading frame of the coding region (reading frame 0 is the default), and maximum length of the self-matches (9 bases is the default). An example script is `test_dna2prot_12.pl`.

```
use Dna2prot;
my $dna  =
'GGGTCGACGTCCGGTTCTACTTCCGGAACTAGCGGGTCAGGAGGAGGATCAACTGGCTCTGGATCGACCTCAGGTTCA
ACCTCGGGTACTTCCGGATCGACTAGTAGTTCAACAACTACGGGAGGAAGCACTAGCACGGGCTCAACATCGGGAACGA
GCACTACCACAACCTCGGGATCGGGCGGAAGCACAACTGGGTCGGGGGGGTCGACATCCACTAGTGGCGGTAGT';
my $dna2prot =  Dna2prot
             -> new()
             -> find_and_replace_self_matches($dna)
             -> save_dna('test_gst_dna_sequence_self_matches.txt')
             -> save_textbox('test_gst_dna_self_matches_alignment.txt');
```

The output to the textbox file, `test_gst_dna_self_matches_alignment.txt`, is a sequence alignment output showing the locations of the mutations (marked by asterisks).

```
>New_DNA_Sequence
GGGTCGACGTCCGGTTCTACGTCCGGAACTAGCGGGTCAGGAGGAGGATCAACTGGCTCTGGATCGACCTCAGGTTCAA
CGTCGGGTACTTCCGGATCGACTAGTAGTTCAACAACTACGGGAGGAAGCACTAGCACGGGCTCAACATCGGGAACGAG
CACTACCACAACCTCGGGATCGGGCGGAAGCACAACTGGGTCGGGGGGGTCGACATCCACTAGTGGCGGTAGT
>Alignment
.....................*.....................................................
.*..................................................................
.....................................................................
>Original_DNA_Sequence
```

```
GGGTCGACGTCCGGTTCTACTTCCGGAACTAGCGGGTCAGGAGGAGGATCAACTGGCTCTGGATCGACCTCAGGTTCAA
CCTCGGGTACTTCCGGATCGACTAGTAGTTCAACAACTACGGGAGGAAGCACTAGCACGGGCTCAACATCGGGAACGAG
CACTACCACAACCTCGGGATCGGGCGGAAGCACAACTGGGTCGGGGGGGTCGACATCCACTAGTGGCGGTAGT
```

As can be seen, there were two silent mutations made in the original DNA sequence.

DNA Synthesis (<500 bp Scale)

There is interest in implementing a computational bioinformatics tool that allows the design of oligonucleotides to support the development of *in vitro* gene synthesis. Current protocols that make long synthetic DNA molecules rely on the *in vitro* assembly of an array of short oligonucleotides, either by ligase chain reaction (LCR) or by assembly polymerase chain reaction (PCR). (See https://openwetware.org/wiki/DNA_Synthesis_from_Oligos.) Using the method, `dna_synthesis`, you can design oligonucleotides that support LCR or PCR mediated gene assembly. Here is an example script, `test_dna2prot_13.pl`, using assembly PCR.

```
use Dna2prot;
my $dna2prot =  Dna2prot
             -> new();
$dna2prot    -> verbose(1);
$dna2prot    -> open_protein('UBA52.txt')
             -> reverse_translate;
my $dna      = $dna2prot -> current_dna_sequence;
$dna2prot    -> dna_synthesis($dna, 'PCR')
             -> save_textbox('UBA52_synthesis.txt');
```

The script starts out creating a `Dna2prot` object in verbose mode, meaning additional output will be sent to the terminal during execution of the script. Then the script opens the protein in the text file, `UBA52.txt`. A possible DNA encoding the protein sequence is created by `reverse_translate`. The DNA sequence is extracted to the variable, `$dna`, using the property, `current_dna_sequence`. Then the synthesis is performed with `dna_synthesis`, which can take up to three arguments: first the DNA sequence, second the method (either 'PCR' or 'LCR'), and third the target melting temperature, Tm (55 ºC by default). The Tm calculation method is the simple formula where G's and C's are counted as 4 ºC, and A's and T's are counted as 2 ºC. The output to the terminal in verbose mode shows the intermediate construction of the primer hybridization units, their sequence, their locations in the DNA sequence, simple Tm, and length. The output to textbox, which is saved to the file, `UBA52_synthesis.txt`, contains the actual primers to be used in the PCR assembly synthesis reaction. That output is as follows:

```
F1|5'-ATGCAAATCTTTGTCAAGACCCTGACAGGTAAAACTATT-3'
F2|5'-CCCTGACAGGTAAAACTATTACATTAGAGGTCGAGCCG-3'
F3|5'-ACATTAGAGGTCGAGCCGTCTGATACTATTGAAAACGTA-3'
F4|5'-TCTGATACTATTGAAAACGTAAAGGCAAAAATTCAGGACAAA-3'
F5|5'-AAGGCAAAAATTCAGGACAAAGAGGGGATTCCTCCCGA-3'
F6|5'-GAGGGGATTCCTCCCGACCAGCAGCGCTTGATATTT-3'
F7|5'-CCAGCAGCGCTTGATATTTGCTGGCAAGCAGCTAGAG-3'
F8|5'-GCTGGCAAGCAGCTAGAGGATGGGCGCACACTTAGC-3'
F9|5'-GATGGGCGCACACTTAGCGATTACAACATACAGAAAGAG-3'
F10|5'-GATTACAACATACAGAAAGAGAGCACGCTCCATTTGGTAT-3'
R9|5'-TCCCACCTCGGAGTCTTAATACCAAATGGAGCGTGCT-3'
R8|5'-CGCAGTGAAGGTTCTATGATCCCACCTCGGAGTCTTA-3'
R7|5'-ATACTTCTGCGCCAGTTGTCGCAGTGAAGGTTCTATGA-3'
R6|5'-ACATATCATTTTGTCACAGTTATACTTCTGCGCCAGTTGT-3'
R5|5'-AATCTTGCATAACACTTCCGACATATCATTTTGTCACAGTT-3'
R4|5'-TTCACTGCTCGTGGGTGTAATCTTGCATAACACTTCCG-3'
R3|5'-CGCACTTCTTCTTCCTACAATTCACTGCTCGTGGGTGT-3'
R2|5'-CGGAGGTTATTTGTATGTCCGCACTTCTTCTTCCTACAA-3'
```

```
R1|5'-TTTCACCTTCTTTTTCGGGCGGAGGTTATTTGTATGTC-3'
```

The primer names on the left (prior to the pipe) are numbered lowest on the outmost forward ('F' prefix) and reverse ('R' prefix). The reverse primers are already reverse complemented and oriented 5' to 3' for convenient ordering. The primer oligos and associated information are stored as an array of hashes property in the module under `oligos`.

An example of LCR assembly DNA synthesis is shown in the next script, `test_dna2prot_14.pl`.

```
use Dna2prot;
my $dna2prot =  Dna2prot
             -> new();
$dna2prot    -> verbose(1);
$dna2prot    -> open_protein('UBA52.txt')
             -> reverse_translate;
my $dna      = $dna2prot -> current_dna_sequence;
$dna2prot    -> dna_synthesis($dna, 'LCR')
             -> save_textbox('UBA52_synthesis_LCR.txt');
```

The output primer sequences saved from the textbox to file, `UBA52_synthesis_LCR.txt`, is:

```
F1|5'-ATGCAAATATTTGTCAAGACGCTAACCGGGAAGACAATAAC-3'
F2|5'-TTTGGAAGTGGAGCCTTCGGACACCATTGAAAACGTAAAA-3'
F3|5'-GCGAAAATACAGGACAAGGAAGGAATACCTCCTGATCAA-3'
F4|5'-CAGCGCCTCATATTTGCAGGGAAGCAGTTGGAAGACG-3'
F5|5'-GCAGAACATTGTCCGATTATAACATTCAAAAAGAATCGACT-3'
F6|5'-CTTCATTTAGTACTAAGATTACGGGGTGGTATTATCGAAC-3'
F7|5'-CGTCCCTTAGACAATTGGCCCAAAAATATAACTGTGATAA-3'
F8|5'-GATGATTTGTCGAAAATGTTACGCTCGACTACACCCAC-3'
F9|5'-GTGCGGTTAATTGCCGTAAAAAAAAGTGTGGCCATACCA-3'
F10|5'-ATAACTTGCGACCTAAGAAGAAGGTCAAA-3'
R10|5'-CGTCTTGACAAATATTTGCAT-3'
R9|5'-CGAAGGCTCCACTTCCAAAGTTATTGTCTTCCCGGTTAG-3'
R8|5'-CCTTGTCCTGTATTTTCGCTTTTACGTTTTCAATGGTGTC-3'
R7|5'-CTGCAAATATGAGGCGCTGTTGATCAGGAGGTATTCCTT-3'
R6|5'-ATAATCGGACAATGTTCTGCCGTCTTCCAACTGCTTCC-3'
R5|5'-TAATCTTAGTACTAAATGAAGAGTCGATTCTTTTTGAATGTT-3'
R4|5'-GCCAATTGTCTAAGGGACGGTTCGATAATACCACCCCG-3'
R3|5'-TAACATTTTCGACAAATCATCTTATCACAGTTATATTTTTGG-3'
R2|5'-TTACGGCAATTAACCGCACGTGGGTGTAGTCGAGCG-3'
R1|5'-TTTGACCTTCTTCTTAGGTCGCAAGTTATTGGTATGGCCACACTTTTTT-3'
```

Gibson Assembly (>500 bp Scale)

Gibson assembly is a PCR-mediated gene assembly tool to make larger genes from smaller fragments with overlapping ends (usually 15-20 bp overlaps). A larger gene is broken up into ~500 bp fragments with overlapping ends, since 500 bp fragments can be made by chemical synthesis. The method in the module is `Gibson_assembly`, which has two arguments: the target DNA sequence and the target Tm for the overlapping ends. Here is an example script, `test_dna2prot_15.pl`, using Gibson assembly.

```
use Dna2prot;
my $dna2prot =  Dna2prot
             -> new();
$dna2prot    -> verbose(1);
$dna2prot    -> open_dna('PA_DNA.txt')
             -> gibson_assembly
```

```
             -> save_textbox('PA_gibson.txt');
```

The output fasta Gibson sequences saved from the textbox to file, `PA_gibson.txt`, is:

```
>PA_DNA_1-515
TAAATTCTTTTTTATGTTATATATTTATAAAAGTTCTGTTTAAAAAGCCAAAAATAAATAATTATCTCTTTTTATTTAT
ATTATATTGAAACTAAAGTTTATTAATTTCAATATAATATAAATTTAATTTTATACAAAAAGGAGAACGTATATGAAAA
AACGAAAAGTGTTAATACCATTAATGGCATTGTCTACGATATTAGTTTCAAGCACAGGTAATTTAGAGGTGATTCAGGC
AGAAGTTAAACAGGAGAACCGGTTATTAAATGAATCAGAATCAAGTTCCCAGGGGTTACTAGGATACTATTTTAGTGAT
TTGAATTTTCAAGCACCCATGGTGGTTACTTCTTCTACTACAGGGGATTTATCTATTCCTAGTTCTGAGTTAGAAAATA
TTCCATCGGAAAACCAATATTTTCAATCTGCTATTTGGTCAGGATTTATCAAAGTTAAGAAGAGTGATGAATATACATT
TGCTACTTCCGCTGATAATCATGTAACAATGTGGGTAGATG
>PA_DNA_495-1028
ATGTAACAATGTGGGTAGATGACCAAGAAGTGATTAATAAAGCTTCTAATTCTAACAAAATCAGATTAGAAAAAGGAAG
ATTATATCAAATAAAAATTCAATATCAACGAGAAAATCCTACTGAAAAAGGATTGGATTTCAAGTTGTACTGGACCGAT
TCTCAAAATAAAAAAGAAGTGATTTCTAGTGATAACTTACAACTGCCAGAATTAAAACAAAAATCTTCGAACTCAAGAA
AAAAGCGAAGTACAAGTGCTGGACCTACGGTTCCAGACCGTGACAATGATGGAATCCCTGATTCATTAGAGGTAGAAGG
ATATACGGTTGATGTCAAAAATAAAAGAACTTTTCTTTCACCATGGATTTCTAATATTCATGAAAAGAAAGGATTAACC
AAATATAAATCATCTCCTGAAAAATGGAGCACGGCTTCTGATCCGTACAGTGATTTCGAAAAGGTTACAGGACGGATTG
ATAAGAATGTATCACCAGAGGCAAGACACCCCCTTGTGGCAGCTTATCCGATTGTACATG
>PA_DNA_1009-1543
AGCTTATCCGATTGTACATGTAGATATGGAGAATATTATTCTCTCAAAAAATGAGGATCAATCCACACAGAATACTGAT
AGTCAAACGAGAACAATAAGTAAAAATACTTCTACAAGTAGGACACATACTAGTGAAGTACATGGAAATGCAGAAGTGC
ATGCGTCGTTCTTTGATATTGGTGGGAGTGTATCTGCAGGATTTAGTAATTCGAATTCAAGTACGGTCGCAATTGATCA
TTCACTATCTCTAGCAGGGGAAAGAACTTGGGCTGAAACAATGGGTTTAAATACCGCTGATACAGCAAGATTAAATGCC
AATATTAGATATGTAAATACTGGGACGGCTCCAATCTACAACGTGTTACCAACGACTTCGTTAGTGTTAGGAAAAAATC
AAACACTCGCGACAATTAAAGCTAAGGAAAACCAATTAAGTCAAATACTTGCACCTAATAATTATTATCCTTCTAAAAA
CTTGGCGCCAATCGCATTAAATGCACAAGACGATTTCAGTTCTACTCCAATTACAATGAAT
>PA_DNA_1522-2056
TTCTACTCCAATTACAATGAATTACAATCAATTTCTTGAGTTAGAAAAAACGAAACAATTAAGATTAGATACGGATCAA
GTATATGGGAATATAGCAACATACAATTTTGAAAATGGAAGAGTGAGGGTGGATACAGGCTCGAACTGGAGTGAAGTGT
TACCGCAAATTCAAGAAACAACTGCACGTATCATTTTTAATGGAAAAGATTTAAATCTGGTAGAAAGGCGGATAGCGGC
GGTTAATCCTAGTGATCCATTAGAAACGACTAAACCGGATATGACATTAAAAGAAGCCCTTAAAATAGCATTTGGATTT
AACGAACCGAATGGAAACTTACAATATCAAGGGAAAGACATAACCGAATTTGATTTTAATTTCGATCAACAAACATCTC
AAAATATCAAGAATCAGTTAGCGGAATTAAACGTAACTAACATATATACTGTATTAGATAAAATCAAATTAAATGCAAA
AATGAATATTTTAATAAGAGATAAACGTTTTCATTATGATAGAAATAACATAGCAGTTGGG
>PA_DNA_2037-2549
GAAATAACATAGCAGTTGGGGCTGATGAGTCAGTAGTTAAGGAGGCTCATAGAGAAGTAATTAATTCGTCAACAGAGGG
ATTATTGTTAAATATTGATAAGGATATAAGAAAAATATTATCAGGTTATATTGTAGAAATTGAAGATACTGAAGGGCTT
AAAGAAGTTATAAATGACAGATATGATATGTTGAATATTTCTAGTTTACGGCAAGATGGAAAAACATTTATAGATTTTA
AAAAATATAATGATAAATTACCGTTATATATAAGTAATCCCAATTATAAGGTAAATGTATATGCTGTTACTAAAGAAAA
CACTATTATTAATCCTAGTGAGAATGGGGATACTAGTACCAACGGGATCAAGAAAATTTTAATCTTTTCTAAAAAAGGC
TATGAGATAGGATAAGGTAATTCTAGGTGATTTTTAAATTATCTAAAAAACAGTAAAATTAAAACATACTCTTTTTGTA
AGAAATACAAGGAGAGTATGTTTTAAACAGTAATCTAAA
```

These sequences can be accessed from the property, `gibson_sequences`, which is an array of hashes.

```
use Dna2prot;
my $dna2prot =  Dna2prot
             -> new()
             -> open_dna('PA_DNA.txt')
             -> gibson_assembly;
my @sequences = $dna2prot -> gibson_sequences;
#Print a fasta output of Gibson sequences
foreach my $sequence (@sequences) {
  my $seq  = $sequence -> {'SEQUENCE'};
  my $name = $sequence -> {'NAME'};
  print '>' . $name . "\n";
  print $seq . "\n";
};
```

The keys, `SEQUENCE` and `NAME`, access the respective information in the array of hashes. These sequences can be used in `dna_assembly` to synthesize a ~500 bp Gibson fragment from oligonucleotides.

Searching the Protein Library for a match

You can search a protein library containing protein sequences commonly used with `search_protein_library`. The function finds the closest match and determines the point mutations in the searched sequence. It then renames the protein with the point mutations if found using the `protein_name` property. The function `search_protein_library` takes two arguments, `$protein` and `$protein_library_file`, which are the searched protein sequence (or the current protein sequence if left blank) and the filename of the library file containing the commonly used sequences in FASTA format. When the protein library file is left blank, the property set in `protein_library_file` is used to open the protein library. A text file like `protein_library.txt` used the example below shows the FASTA formatting for multiple sequences:

```
>LFN
AGGHGDVGMHVKEKEKNKDENKRKDEERNKTQEEHLKEIMKHIVKIEVKGEEAVKKEAAEKLLEKVPSDVLEMYKAIGG
KIYIVDGDITKHISLEALSEDKKKIKDIYGKDALLHEHYVYAKEGYEPVLVIQSSEDYVENTEKALNVYYEIGKILSRD
ILSKINQPYQKFLDVLNTIKNASDSDGQDLLFTNQLKEHPTDFSVEFLEQNSNEVQEVFAKAFAYYIEPQHRDVLQLYA
PEAFNYMDKFNEQEINLSLEELKDQ

>PA
MKKRKVLIPLMALSTILVSSTGNLEVIQAEVKQENRLLNESESSSQGLLGYYFSDLNFQAPMVVTSSTTGDLSIPSSEL
ENIPSENQYFQSAIWSGFIKVKKSDEYTFATSADNHVTMWVDDQEVINKASNSNKIRLEKGRLYQIKIQYQRENPTEKG
LDFKLYWTDSQNKKEVISSDNLQLPELKQKSSNSRKKRSTSAGPTVPDRDNDGIPDSLEVEGYTVDVKNKRTFLSPWIS
NIHEKKGLTKYKSSPEKWSTASDPYSDFEKVTGRIDKNVSPEARHPLVAAYPIVHVDMENIILSKNEDQSTQNTDSQTR
TISKNTSTSRTHTSEVHGNAEVHASFFDIGGSVSAGFSNSNSSTVAIDHSLSLAGERTWAETMGLNTADTARLNANIRY
VNTGTAPIYNVLPTTSLVLGKNQTLATIKAKENQLSQILAPNNYYPSKNLAPIALNAQDDFSSTPITMNYNQFLELEKT
KQLRLDTDQVYGNIATYNFENGRVRVDTGSNWSEVLPQIQETTARIIFNGKDLNLVERRIAAVNPSDPLETTKPDMTLK
EALKIAFGFNEPNGNLQYQGKDITEFDFNFDQQTSQNIKNQLAELNVTNIYTVLDKIKLNAKMNILIRDKRFHYDRNNI
AVGADESVVKEAHREVINSSTEGLLLNIDKDIRKILSGYIVEIEDTEGLKEVINDRYDMLNISSLRQDGKTFIDFKKYN
DKLPLYISNPNYKVNVYAVTKENTIINPSENGDTSTNGIKKILIFSKKGYEIGZ

>LF
MNIKKEFIKVISMSCLVTAITLSGPVFIPLVQGAGGHGDVGMHVKEKEKNKDENKRKDEERNKTQEEHLKEIMKHIVKI
EVKGEEAVKKEAAEKLLEKVPSDVLEMYKAIGGKIYIVDGDITKHISLEALSEDKKKIKDIYGKDALLHEHYVYAKEGY
EPVLVIQSSEDYVENTEKALNVYYEIGKILSRDILSKINQPYQKFLDVLNTIKNASDSDGQDLLFTNQLKEHPTDFSVE
FLEQNSNEVQEVFAKAFAYYIEPQHRDVLQLYAPEAFNYMDKFNEQEINLSLEELKDQRMLARYEKWEKIKQHYQHWSD
SLSEEGRGLLKKLQIPIEPKKDDIIHSLSQEEKELLKRIQIDSSDFLSTEEKEFLKKLQIDIRDSLSEEEKELLNRIQV
DSSNPLSEKEKEFLKKLKLDIQPYDINQRLQDTGGLIDSPSINLDVRKQYKRDIQNIDALLHQSIGSTLYNKIYLYENM
NINNLTATLGADLVDSTDNTKINRGIFNEFKKNFKYSISSNYMIVDINERPALDNERLKWRIQLSPDTRAGYLENGKLI
LQRNIGLEIKDVQIIKQSEKEYIRIDAKVVPKSKIDTKIQEAQLNINQEWNKALGLPKYTKLITFNVHNRYASNIVESA
YLILNEWKNNIQSDLIKKVTNYLVDGNGRFVFTDITLPNIAEQYTHQDEIYEQVHSKGLYVPESRSILLHGPSKGVELR
NDSEGFIHEFGHAVDDYAGYLLDKNQSDLVTNSKKFIDIFKEEGSNLTSYGRTNEAEFFAEAFRLMHSTDHAERLKVQK
NAPKTFQFINDQIKFIINS
```

Here is a script (`test_dna2prot_18.pl`) to rename a mutant PA sequence:

```
use Dna2prot;
my $dna2prot =  Dna2prot
             -> new();
$dna2prot    -> verbose(1);
$dna2prot    -> open_record('PA_reverse_translate_record.txt')
```

```
            -> find_largest_orf
            -> site_directed_mutagenesis(70, 'TYR')
            -> transfer_mutant_to_current
            -> search_protein_library(undef, 'protein_library.txt')
            -> save_record('PA_G70Y.txt');
```

You can also add a new sequence to the protein library with `append_protein_library`, which takes three arguments, `$protein_sequence, $protein_name, $file`. Protein sequence and protein name are self-explanatory; the file is the protein library file. The file is automatically saved since it calls the `save_protein_library` method (which when called takes the protein library filename as its only argument.

Here is the truncated output in the record file (`PA_G70Y.txt`) from the above script:

```
# Protein name: PA_G70Y
# Protein length: 764 residues
# Molecular weight: 85944.3 Da
# Isoelectric point (pI): 5.89
# Extinction coefficient: 81710 M-1 cm-1

>PA_G70Y
MKKRKVLIPLMALSTILVSSTGNLEVIQAEVKQENRLLNESESSSQGLLGYYFSDLNFQAPMVVTSSTTYDLSIPSSEL
ENIPSENQYFQSAIWSGFIKVKKSDEYTFATSADNHVTMWVDDQEVINKASNSNKIRLEKGRLYQIKIQYQRENPTEKG
LDFKLYWTDSQNKKEVISSDNLQLPELKQKSSNSRKKRSTSAGPTVPDRDNDGIPDSLEVEGYTVDVKNKRTFLSPWIS
NIHEKKGLTKYKSSPEKWSTASDPYSDFEKVTGRIDKNV…

>PA_DNA_G70Y
ATGAAAAAACGTAAGGTCCTGATTCCATTGATGGCTCTGTCAACCATCCTGGTGAGTTCGACAGGCAATCTGGAAGTAA
TACAGGCTGAGGTCAAGCAGGAGAACCGGCTGCTAAATGAATCTGAGTCTAGTTCTCAGGGTCTTCTGGGCTATTACTT
CAGTGATCTAAATTTTCAGGCACCGATGGTTGTGACCTCGTCGA…
```

Notice the protein name is PA_G70Y since PA protein was detected as the best match and it had the point mutation G70Y.